Skifer Atlas — Brief de conception (avec maquettes)

Statut : conception initiale complète, écrans définis. Version 1.2 — 29 octobre 2026, maquettes v1 du 30 octobre 2026. Le module s'est appelé « Skifer Annuaire » pendant la conception ; renommé Skifer Atlas le 29 octobre 2026.

1. Contexte et problème

Des collègues sont intéressés par Skifer, mais ce qui les bloque est structurel avant d'être technique :

- personne n'a de définition partagée des KPI ;
- les entités d'un même groupe ne calculent pas un KPI de la même façon ;
- leurs outils et leurs sources sont hétérogènes (ERP, Snowflake, fichiers Excel), avec des formules qui divergent.

Avant d'optimiser du code ou de générer une couche sémantique, il faut donc un travail de gestion de la connaissance : un point central qui fasse foi pour les définitions.

2. Positionnement dans l'écosystème Skifer

Skifer est pensé comme une librairie, un moteur central, autour duquel gravitent des modules :

- **Skifer pour Databricks** : semantic layer, metric views, agent builder de backup ;
- **Skifer Board** : dashboards ;
- **Skifer Atlas** : définitions des concepts métier et des KPI (ce brief).

Atlas est un module séparé. Il ne remplace pas l'arbitrage organisationnel (qui possède un KPI, quelle définition l'emporte), mais il l'outille et le déclenche en rendant les divergences visibles.

3. Principes structurants

3.1 Le modèle canonique, pièce centrale

- Un modèle canonique partagé par tous, qui couvre **les concepts métier** (client actif, commande, date de référence…) **et les KPI** construits sur ces concepts.
- Des **déclinaisons par entité** rattachées au modèle canonique (voir section 4).
- **Immuabilité par version** : on ne modifie pas une définition à la légère. Toute évolution crée une nouvelle version, les anciennes restent consultables. Lien direct avec l'historique de certification auditable de Skifer.

3.2 Séparer le « quoi » et le « comment »

- **Quoi** : la définition métier canonique.
- **Comment** : les implémentations rattachées (DAX Power BI, SQL sur les plateformes connectées).
- Distinguer une **variante voulue** (par exemple un calendrier fiscal propre à une entité) d'un **écart accidentel**.

3.3 La définition sous forme de graphe, pas de texte libre

- Aucune place à l'interprétation : pas de grand champ de texte libre.
- La définition est un **graphe orienté sans cycle** : les nœuds sont des opérations, les arêtes portent l'ordre d'application (les priorités).
- **Vocabulaire fermé** de types de nœuds et d'opérations. Une interface drag and drop façon ETL n'est qu'un éditeur visuel par-dessus.
- **Exigence de déterminisme** : appliquer techniquement le graphe doit toujours donner exactement le même résultat.

3.4 Format pivot : le YAML Skifer

- Le graphe est sérialisé dans **le YAML actuel du moteur Skifer** : pas de nouveau format, pour ne pas complexifier la structure. Un seul format pour tous les modules.
- Ajouts : un **identifiant stable par nœud** (diffs Git lisibles, surcharges adressables) et un **numéro de version du schéma**.
- Un **schéma de validation** rejette tout YAML hors vocabulaire.

3.5 Types de nœuds (vocabulaire fermé)

1. **Source** — déclare le format et le fuseau d'origine de ses colonnes temporelles
2. **Filtre** — déclare explicitement s'il garde ou non les valeurs nulles
3. **Jointure** — clés, type et cardinalité attendue déclarés
4. **Union**
5. **Calcul de colonne**
6. **Dédoublonnage** — source classique d'écarts entre entités
7. **Agrégation** — granularité explicite
8. **Opération temporelle** — fenêtres, calendriers, conversions de fuseau et de format (toute conversion est un nœud explicite)
9. **Référence à un KPI** — construire un KPI sur un autre déjà défini (par exemple une marge sur le revenu net, ou la déduction des taxes écrite une seule fois et rappelée partout). Forme aussi un graphe de dépendances : quand un KPI change de version, on sait quels KPI sont touchés.
10. **Opération entre métriques agrégées** — ratio, différence… avec ordre calcul/agrégation explicite (un ratio de sommes n'est pas une moyenne de ratios).

Les branches parallèles ne se rejoignent que par une jointure ou une union explicite.

3.6 Sémantique figée

- **Valeurs nulles** : comportement SQL dans les agrégations (ignorées) ; chaque filtre déclare s'il garde ou non les nulls, car « pays différent de France » exclut aussi les lignes sans pays, source fréquente d'écart.
- **Jointures** : type et cardinalité attendue obligatoires. **Contrôle de cardinalité à l'exécution, requis par défaut** : si la jointure multiplie des lignes au-delà de la cardinalité déclarée, l'exécution échoue plutôt que de produire un résultat faux. **Bypass possible au cas par cas**, visible dans le graphe et remonté en comparaison, pour qu'un écart ne puisse pas se cacher derrière un contrôle désactivé.

- **Temps** : chaque KPI déclare un **calendrier de référence** (civil ou fiscal) et un **fuseau d'évaluation** ; les deux sont des points de variation déclarés. Chaque source déclare le **format et le fuseau d'origine** de ses colonnes temporelles ; toute transformation de format ou de fuseau appliquée par une équipe est un nœud explicite du graphe, donc visible en comparaison (cas typique : un timestamp converti en date locale par une équipe et pas par l'autre, qui fait changer de jour les commandes de fin de journée).
- **Arrondis** : jamais dans le graphe, uniquement à la restitution.

4. Variantes par entité

Deux modes coexistent, parce que dans les organisations où les outils divergent fortement d'une entité à l'autre, la surcharge n'est pas toujours possible.

4.1 Variante par surcharge

- La variante n'est pas une copie du graphe : c'est le **graphe canonique + une liste de surcharges déclarées** sur des nœuds précis.
- Chaque surcharge porte sa justification et sa validation.
- En comparaison, tout écart couvert par une surcharge déclarée est normal ; tout autre écart est une divergence.

Points de variation :

- Le canonique **déclare à l'avance les nœuds surchargeables** (par exemple calendrier, fuseau, devise, déduction des taxes).
- Une surcharge sur un point de variation déclaré passe avec la **validation légère** des déclinaisons.
- Une surcharge **hors des points déclarés** reste possible, mais exige la **validation de niveau canonique** et est signalée comme écart à examiner.
- Si une même surcharge hors points déclarés revient souvent, c'est le signal qu'il faut ajouter un point de variation au canonique.

Exemple — revenu net : le canonique déduit les taxes du revenu ; « déduction des taxes » est un point de variation déclaré, et chaque pays le surcharge avec ses propres taxes.

4.2 Variante from scratch

- Pour les cas où la surcharge n'est pas réalisable : la variante est un graphe complet construit indépendamment.
- Elle reste **rattachée au canonique par ses concepts**, donc comparable structurellement et en chiffres, mais sans liste d'écarts autorisés préétablie.
- **Justification obligatoire**, et la variante est signalée **« candidate à la convergence »** : elle a vocation, avec le temps, à devenir une surcharge du canonique.

Intérêt : rendre visibles les différences entre entités, dans un contexte où l'on parle beaucoup d'unification. Mettre en avant la divergence pour tendre vers la convergence.

5. Gouvernance

5.1 Rôles

- **Contributeurs** : ouvrent des propositions de modification (une pull request côté Git).
- **Validateurs** : font la revue et acceptent ou refusent les changements.
- **Référents** : dont le **référent d'annuaire**, qui paramètre la gouvernance.

En démarrage à froid, ce fonctionnement permet de mettre en place le processus et d'impliquer aussi bien les consultants que les clients, pour qu'ils prennent conscience des chantiers à mener dans leur entreprise.

5.2 Nombre de validateurs paramétrable

- Le nombre de validateurs requis est un **paramètre**, avec un **minimum de 1**.
- Il est réglé par le référent selon le contexte : certaines entreprises auront du mal à trouver plusieurs points de contact, d'autres non.
- Objectif : ne pas imposer d'emblée plusieurs validateurs, ce qui pourrait bloquer l'adoption de l'outil.
- Piste : deux valeurs distinctes, plus exigeante pour le canonique, plus légère pour les déclinaisons locales.

5.3 Multiprojet : espaces de travail et domaines

Deux niveaux distincts, qui répondent à deux besoins différents.

Espace de travail — un par client

- Chaque espace possède **son dépôt Git, ses utilisateurs, sa gouvernance** (rôles, nombre de validateurs) et ses connexions.
- **Étanchéité totale** : aucun KPI, aucun concept, aucune connexion ne traverse d'un espace à l'autre.
- Un consultant travaillant pour plusieurs clients **bascule d'un espace à l'autre** par un sélecteur, à la manière de Databricks ou GitHub.

Domaine métier — à l'intérieur d'un espace

- Finance, ventes, supply… Le domaine est un **attribut des concepts et des KPI**, filtrable dans le catalogue.
- Chaque domaine peut avoir **ses référents et ses validateurs**.
- Le **modèle canonique reste unique** dans l'espace : un KPI finance peut référencer un concept ventes. Pas d'étanchéité entre domaines, c'est un découpage d'organisation, pas de cloisonnement.

6. Deux modes, une même source de vérité

Atlas est la source de vérité ; les metric views et les business rules en sont des produits compilés.

- **Mode extraction (priorité n°1)** — pour l'existant : Skifer analyse les implémentations en place, les traduit dans le graphe canonique et les compare.
- **Mode construction** — pour les nouveaux projets : la définition sert de contrat. Skifer en génère les metric views, les business rules et des tests sur les tables. Ce sont les « rails » posés sur la construction des tables et des règles.

6.1 Parseurs d'extraction

- **Temps 1 : SQL.** Cohérent avec les premiers connecteurs (Databricks, Snowflake) et avec la tendance à disposer de tables gold prêtes à consommer, même si la pratique n'est pas encore la plus répandue. SQLGlot fournit l'arbre syntaxique à traduire en graphe.
- **Temps 2 : DAX et Power Query.** À ne pas oublier : une partie de la logique vit dans les mesures DAX, et une autre dans le Power Query Editor. Power Query est un problème à part (un « graphe dans le graphe »). Piste : le format projet Power BI (PBIP) enregistre le modèle en texte, mesures DAX et requêtes Power Query comprises.

7. Accès aux données : centralisation logique

- La centralisation physique (tout rapatrier dans un lakehouse, ou un access layer unique) est jugée quasi impossible, notamment pour des raisons politiques entre entités.
- Option retenue : **centralisation logique**. Les données restent où elles sont, Atlas tient un **registre des points d'accès**.
- Chaque déclinaison d'un KPI pointe vers sa source réelle (plateforme, schéma, table), avec la connexion, le propriétaire et la fraîcheur des données.
- Le graphe Skifer en Delta devient la carte de tout le patrimoine, au-delà de ce qui est dans Unity Catalog.

7.1 Connecteurs : top 5 des plateformes

Plateforme	Authentification interactive (outil natif)
Databricks	OAuth avec le compte de l'utilisateur, ou token personnel
Snowflake	SSO via le fournisseur d'identité de l'entreprise, ou paire de clés
BigQuery (GCP)	OAuth avec le compte Google de l'utilisateur
Microsoft Fabric	Entra ID (SSO Microsoft)
Amazon Redshift	Fédération d'identité via IAM

- **Ordre d'implémentation : Databricks et Snowflake en premier.** Deux dialectes dès le départ, ce qui valide tout de suite la compilation multi-plateforme et la comparaison entre sources.
- BigQuery, Fabric et Redshift ensuite.
- Les ERP (SAP, Oracle…) sont généralement déversés dans l'une de ces plateformes : pas de connexion directe aux ERP au départ.
- SQLGlot couvre les dialectes SQL de ces cinq plateformes.

7.2 Droits d'accès : priorité au mode interactif

- **Périmètre de cette version : mode interactif uniquement.** Le mode planifié viendra beaucoup plus tard.
- Chaque comparaison s'exécute **avec les droits de la personne qui la lance**. Pas de « super compte technique » à faire accepter aux entités.
- **Coffre de secrets personnel, jamais partagé** : les tokens d'accès sont propres à chaque utilisateur, stockés chiffrés sur sa machine (dans l'esprit de DuckDB en local), jamais sur un serveur ni dans Git. Chaque utilisateur voit et révoque ses propres connexions ; le référent ne gère que la liste des plateformes autorisées, jamais les tokens. Les tokens sont rangés **par espace de travail** : une connexion Databricks d'un client ne sert jamais pour un autre.
- *Plus tard — mode planifié* : il faudra un compte de service accordé par chaque entité.

8. Comparaison et réconciliation

8.1 Deux niveaux

- **Comparaison structurelle** : Atlas compare des graphes, entièrement à partir de ce qui est versionné dans Git.
- **Comparaison chiffrée** : les définitions sont exécutées sur les données réelles pour mesurer les écarts de résultats.
- Même à **sources différentes**, la comparaison structurelle reste possible si chaque source est rattachée aux mêmes concepts canoniques.

8.2 Algorithme de comparaison de graphes

1. **Normaliser** chaque graphe (règles en 8.2.1). Formes normalisées identiques = pas de divergence (comparaison d'empreintes).
2. **Aligner et comparer** : apparier les nœuds par type, concept et position, relever les nœuds ajoutés, retirés ou modifiés, et les changements d'ordre significatifs.
3. **Classer** l'écart :
   - couvert par une **surcharge sur point de variation déclaré** → normal ;
   - **surcharge hors points déclarés** → à examiner ;
   - écart d'une **variante from scratch** → justifié, à suivre pour convergence ;
   - **divergence critique** : filtre, granularité ou fonction d'agrégation ;
   - **divergence cosmétique** : libellé, nommage.

8.2.1 Règles de normalisation (version simple, à enrichir à l'usage)

- **Seuls les filtres successifs sur un même flux** — entre deux opérations structurantes (jointure, agrégation, dédoublonnage, opération temporelle…) — sont considérés commutatifs : ils sont fusionnés et triés dans un ordre canonique. Le graphe se lit ainsi comme une suite de sous-modèles délimités par les opérations structurantes.
- **Tout autre ordre est conservé tel quel** : un changement d'ordre entre opérations structurantes remonte comme divergence, quitte à être trop prudent.
- Conséquence : deux graphes qui ne diffèrent que par l'ordre de leurs filtres sont reconnus identiques.
- Une normalisation plus poussée sera envisagée à l'usage, si le besoin se présente.

Limite : l'équivalence de deux formules ne peut pas être prouvée dans tous les cas ; l'écart est alors marqué « à vérifier » et passe en comparaison chiffrée.

8.3 Comparaison chiffrée des résultats

Décision assumée : aucune donnée issue d'Excel ou d'un tableur n'entre dans la comparaison de résultats.

Principe : comparer des résultats sur un **scope déclaré** — même périmètre, même période, même granularité — déclaré dans Atlas avec le KPI.

Répartition des rôles : Atlas **orchestre**, le moteur Skifer **exécute**.

1. Connexion aux plateformes via le registre des points d'accès.
2. Le moteur **compile le graphe** de chaque déclinaison et le graphe canonique dans le **dialecte SQL de chaque plateforme** (SQLGlot).
3. **Exécution en push-down**, là où se trouvent les données.
4. **Seuls des résultats agrégés** sont rapatriés : aucune donnée détaillée ne sort de chez une entité.
5. Mise au même format (KPI, version, déclinaison, dimensions, valeur), jointure sur les dimensions, écart en pourcentage.

Tolérance aux deux niveaux : globale (scores comparables) et par KPI (écarts justifiés par des contraintes fonctionnelles, voire culturelles).

Pourcentage de validation : part des cellules du scope dans la tolérance, accompagnée de l'**écart maximal**.

Restitution (pistes) : matrice de synthèse colorée ; détail par dimension ; lien vers le nœud divergent identifié par la comparaison de graphes.

8.4 Stockage des résultats de comparaison

- **Décision : DuckDB en local, d'abord.** Dans l'esprit de DAX Studio : les opérations tournent sur la machine de la personne, rien n'est stocké ailleurs. Base embarquée, sans installation, ce qui accélère le déploiement de l'outil et simplifie l'adoption chez les entités (rien n'est hébergé nulle part). Une base DuckDB **par espace de travail**, pour que les résultats d'un client ne se mélangent jamais avec ceux d'un autre.
- **Plus tard, à l'usage** : un mode hébergé ou partagé conservant les extracts pour audit, si le besoin se confirme.

Règle de conservation : seules les **données extraites des bases** sont supprimées une fois la comparaison enregistrée. Tout ce qui est versionné, le scope, les requêtes exécutées et l'ensemble des résultats d'analyse sont conservés.

9. Accès et stockage des définitions

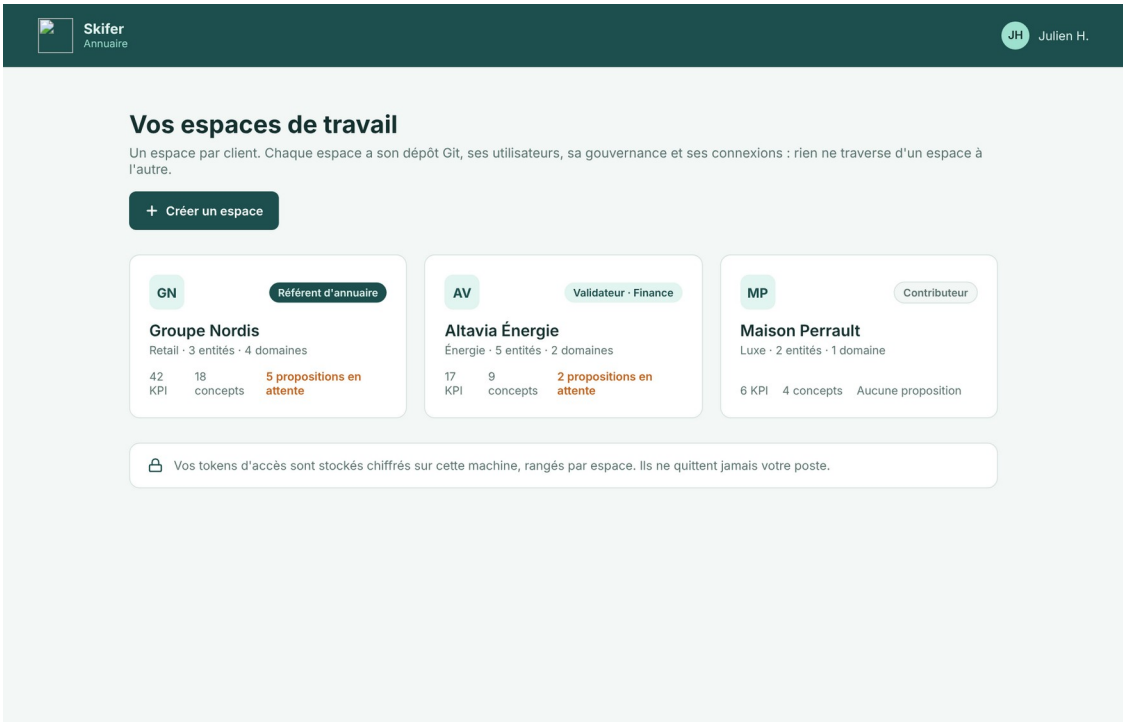
- Accès via un **applicatif externe**.
- **Définitions canoniques stockées sur Git**, un dépôt par espace de travail : une pull request = une proposition de nouvelle version, la revue = l'arbitrage par les validateurs.

- Dans l'applicatif, **une sauvegarde crée une pull request et une demande de validation** : les utilisateurs métier ne manipulent jamais Git.
- **Base de données** : index de recherche reconstructible depuis Git, jamais une source de vérité.

10. Écrans et fonctionnalités de l'application

Base pour les premières maquettes (réalisées dans Claude Design le 29 octobre 2026). Chaque écran est rattaché aux principes qui le justifient, pour garder la finalité de chaque fonctionnalité derrière un bouton.

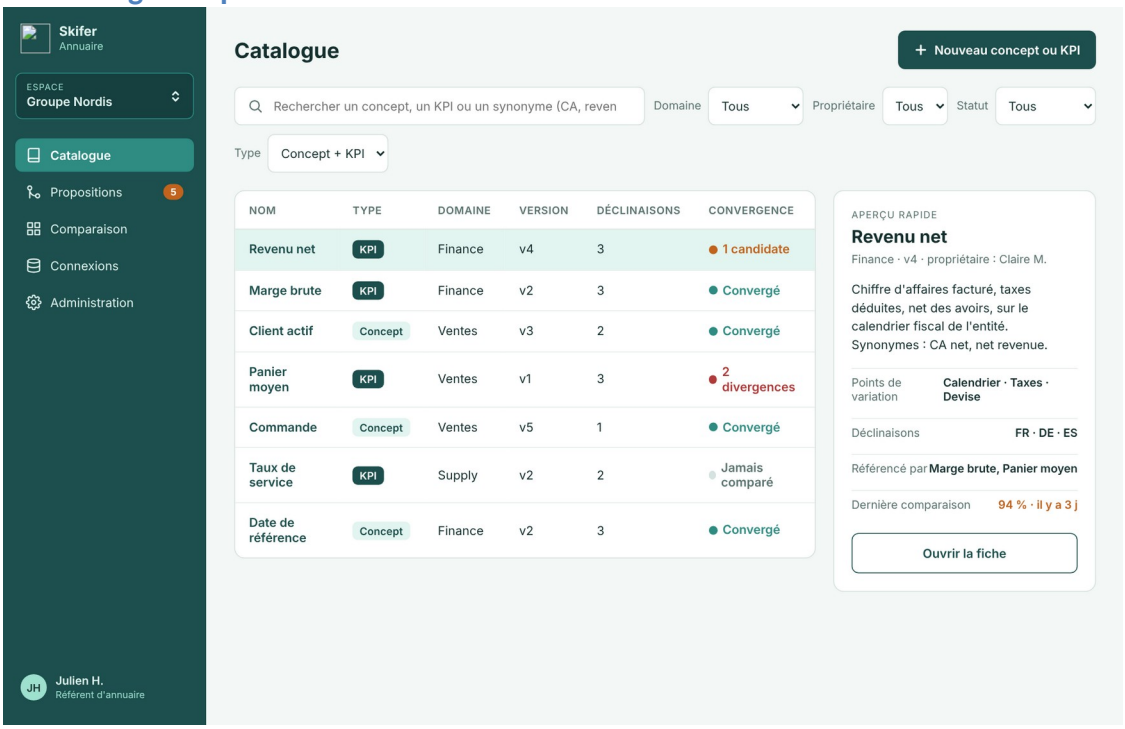
10.0 Éléments transverses



Maquette 10.0

- **Sélecteur d'espace de travail** : en permanence visible (en-tête ou barre latérale), il affiche l'espace courant et permet de basculer vers un autre. Tout le reste de l'application est contextualisé par l'espace sélectionné : catalogue, propositions, comparaisons, connexions, administration.
- **Page d'accueil des espaces** : à l'ouverture, liste des espaces auxquels l'utilisateur a accès, avec son rôle dans chacun ; création d'un espace réservée aux référents.

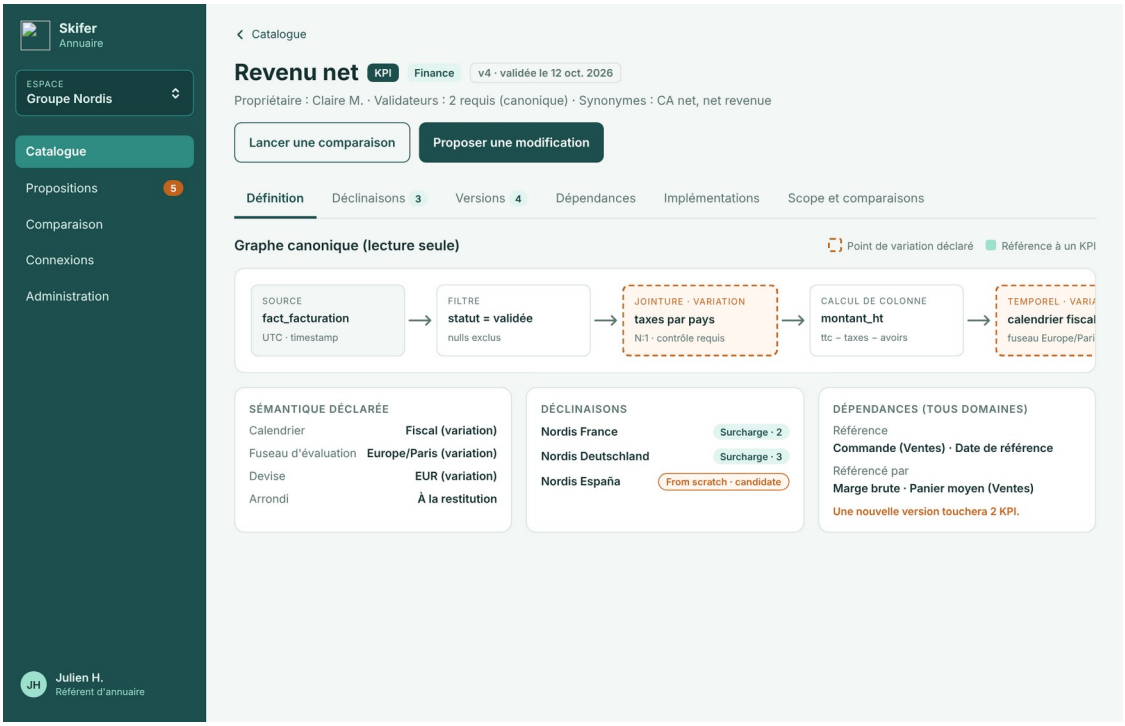
10.1 Catalogue — porte d'entrée



Maquette 10.1

- **Barre de recherche**, qui cherche aussi dans les synonymes.
- **Filtres** : **domaine métier**, propriétaire, statut, type (concept ou KPI).
- **Liste en tableau** : nom, **domaine**, version courante, nombre de déclinaisons, indicateur de convergence.
- **Panneau latéral d'aperçu rapide** au clic, avant d'ouvrir la fiche complète.
- Bouton **« nouveau concept ou KPI »**, réservé aux contributeurs ; le domaine est renseigné à la création.

10.2 Fiche KPI — cœur de l'outil



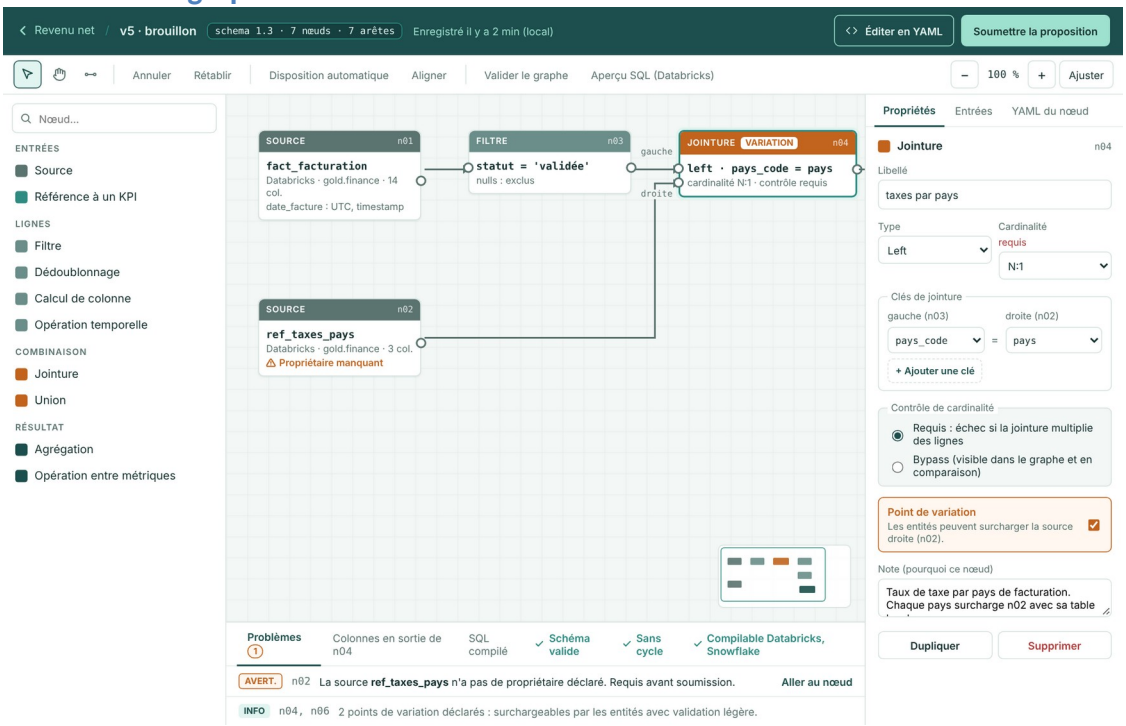
Maquette 10.2

En-tête : nom, domaine, propriétaire, version courante.

Organisée en onglets :

- **Définition** : graphe canonique en lecture, points de variation mis en évidence.
- **Déclinaisons** : liste par entité, mode (surcharge ou from scratch), justification, badge « candidate à la convergence ».
- **Versions** : historique immuable, diff entre deux versions.
- **Dépendances** : KPI référencés et KPI qui le référencent, y compris ceux d'autres domaines (le canonique est unique dans l'espace).
- **Implémentations** : le « comment », SQL ou DAX rattachés par plateforme.
- **Scope et comparaisons** : dernières comparaisons chiffrées.
- Bouton **« proposer une modification »**.

10.3 Éditeur de graphe

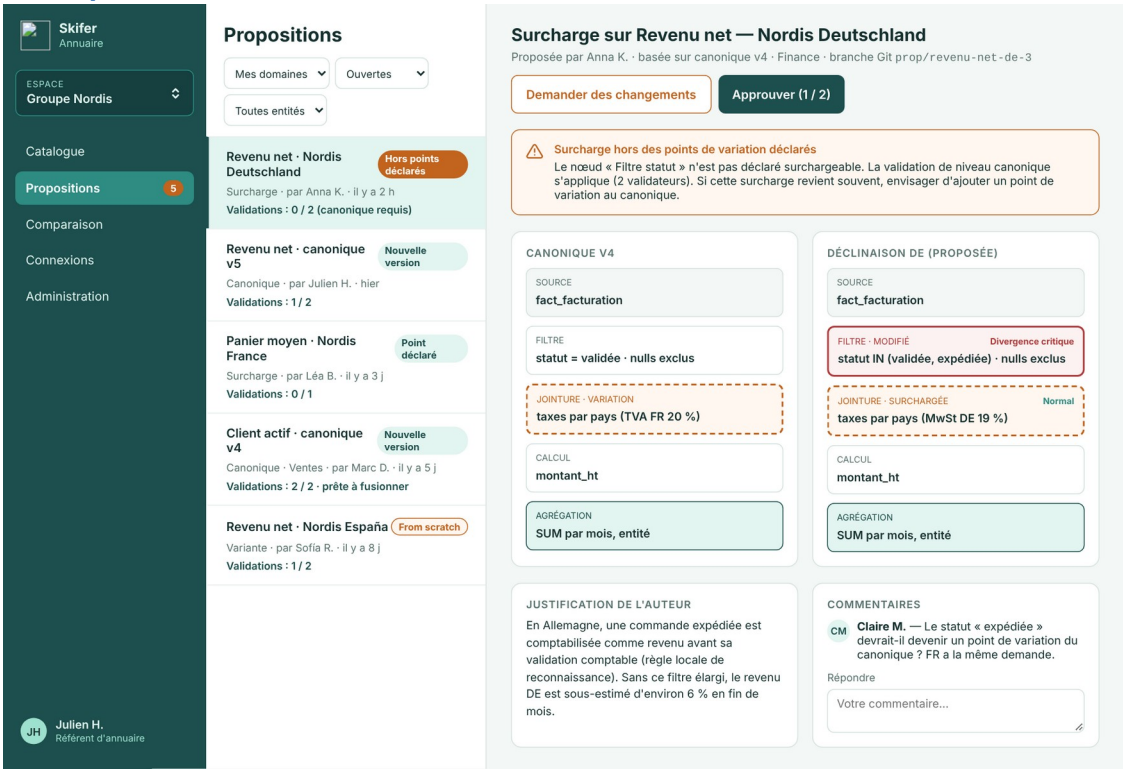


Maquette 10.3

Outil de DAG fonctionnel, pas une illustration :

- **Barre d'outils** : sélection, déplacement de la vue, ajout d'arête, annuler/rétablir, disposition automatique, validation du graphe, aperçu SQL par plateforme, zoom.
- **Palette à gauche** : les dix types de nœuds, regroupés par famille (entrées, lignes, combinaison, résultat), avec recherche. C'est le vocabulaire fermé, rien d'autre n'est disponible.
- **Canevas au centre** : nœuds compacts avec en-tête coloré par famille, identifiant stable, expression en monospace, ports d'entrée/sortie visibles (deux ports nommés pour une jointure), arêtes orthogonales ; marquage visible des points de variation ; avertissements affichés sur le nœud concerné ; mini-carte.
- **Panneau de propriétés à droite** (onglets Propriétés / Entrées / YAML du nœud) : champs obligatoires selon le type de nœud (nulls pour un filtre, cardinalité et bypass pour une jointure, fuseau et format pour une source) ; les clés de jointure se choisissent parmi les colonnes réellement disponibles en amont.
- **Panneau bas** : Problèmes, Colonnes en sortie du nœud sélectionné, SQL compilé ; état de validation (schéma, cycles, compilabilité) en permanence.
- **Bouton « éditer en YAML »** : bascule en plein écran sur le YAML, avec la même validation de schéma, puis retour au canevas (comme la vue YAML des jobs Databricks). Pas de vue YAML en parallèle : le YAML reste le pivot, le canevas n'est qu'une vue dessus.
- **« Soumettre la proposition »** crée la pull request.

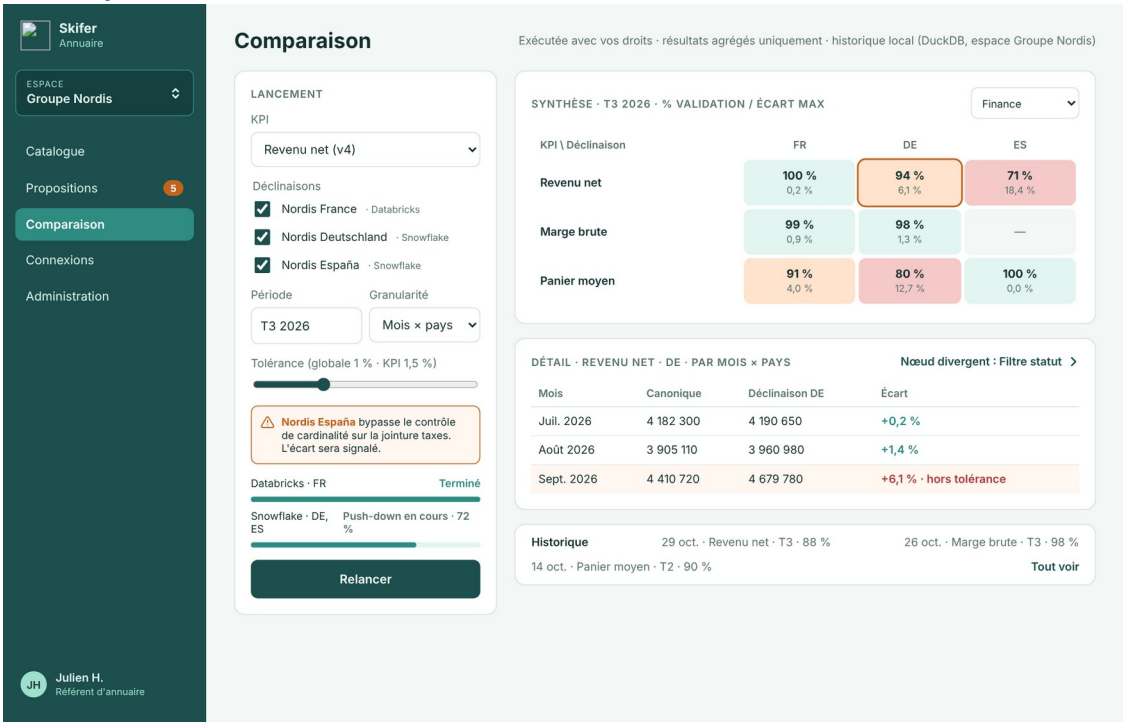
10.4 Propositions



Maquette 10.4

- **File** : propositions ouvertes, filtrables par **domaine**, KPI, auteur, entité, statut ; validations obtenues sur validations requises. Un validateur voit en priorité les propositions de ses domaines.
- **Détail d'une proposition** : diff visuel des deux graphes côte à côte, nœuds ajoutés, retirés ou modifiés surlignés, classification de l'écart, justification de l'auteur, fil de commentaires, boutons « approuver » et « demander des changements ».
- Mention claire quand une surcharge sort des points déclarés (validation canonique exigée).

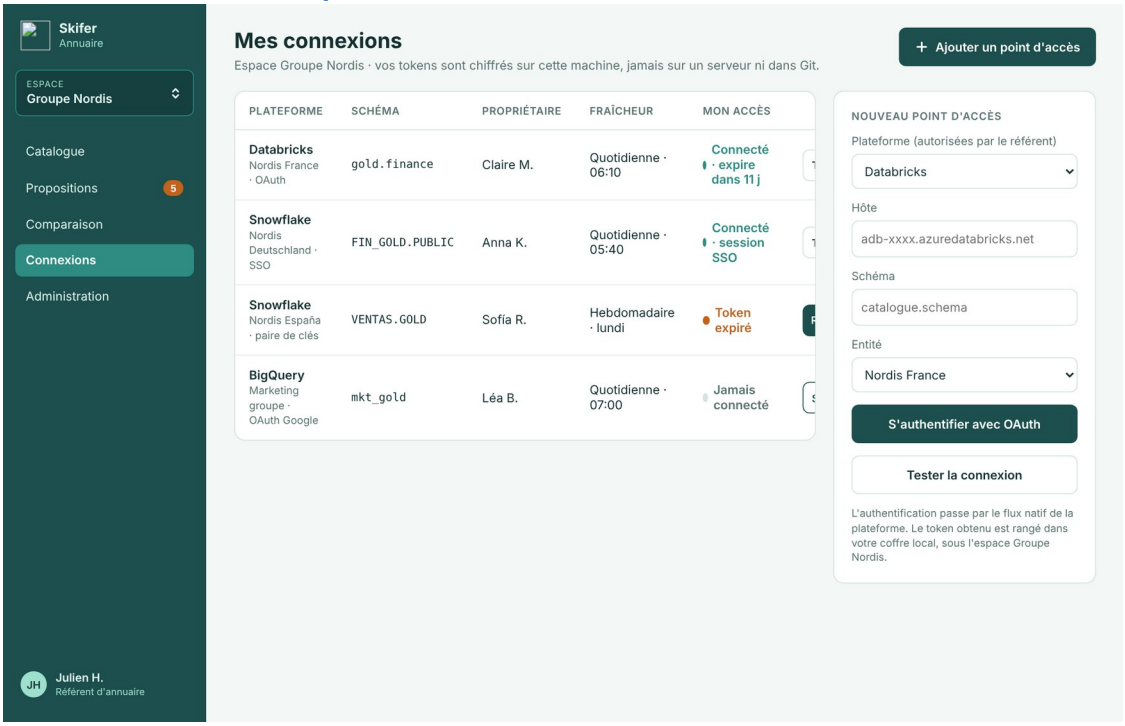
10.5 Comparaison



Maquette 10.5

- **Lancement** : choix du KPI, des déclinaisons, du scope (période, périmètre, granularité) et de la tolérance ; connexion avec les droits de l'utilisateur ; progression par plateforme ; alerte si un contrôle de cardinalité est bypassé.
- **Synthèse** : matrice colorée déclinaisons par KPI, pourcentage de validation et écart maximal dans chaque cellule ; filtrable par domaine.
- **Détail** : au clic sur une cellule, tableau par dimension, écarts hors tolérance en évidence, lien direct vers le nœud divergent identifié par la comparaison structurelle.
- **Historique des comparaisons**, stocké en local dans DuckDB, une base par espace.

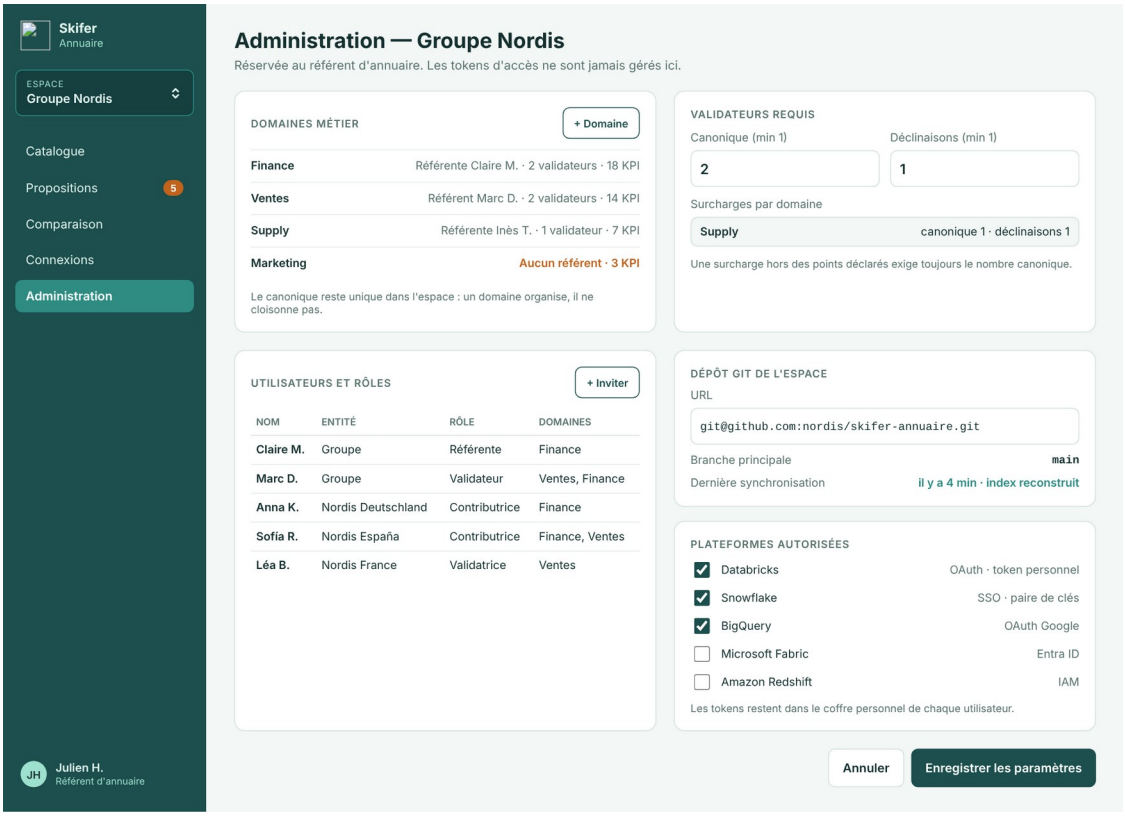
10.6 Connexions — écran personnel



Maquette 10.6

- Liste des points d'accès enregistrés **dans l'espace courant** : plateforme, schéma, propriétaire, fraîcheur, état de la connexion pour l'utilisateur courant.
- Formulaire d'ajout par plateforme, avec le flux d'authentification natif (OAuth, SSO).
- Bouton « tester la connexion ».
- Chaque utilisateur voit uniquement ses propres connexions et peut révoquer ses tokens (coffre personnel, rangé par espace, voir 7.2).

10.7 Administration — réservée au référent



Maquette 10.7

Paramètres de l'espace courant :

- Gestion des utilisateurs et de leurs rôles, par entité et **par domaine**.
- **Liste des domaines** de l'espace : création, renommage, référents et validateurs associés.
- Nombre de validateurs requis, avec les deux valeurs canonique et déclinaisons (surchargeable par domaine si besoin).
- Dépôt Git lié à l'espace.
- Liste des plateformes autorisées (jamais les tokens).

11. Pistes complémentaires (sans priorité pour l'instant)

- **Tables de référence de paramètres** (par exemple les taxes par pays) : la variation devient un simple paramètre, le graphe reste identique partout et la comparaison structurelle passe directement. À traiter plus tard : les clients n'ont en général pas encore ce niveau de maturité.
- **Consommateurs** : quels dashboards, rapports ou agents utilisent chaque KPI, pour mesurer l'impact d'une nouvelle version.
- **Journal des décisions** : pourquoi une définition a été retenue, par qui, et quelles alternatives ont été écartées.
- **Valeurs de référence** : chiffre attendu sur une période connue, servant de cas de test.

- **Synonymes** : « CA », « chiffre d'affaires », « revenue » ; utile pour Genie ou l'agent Skifer.

12. Points volontairement reportés

Confirmés comme reportés après revue complète (29 octobre 2026) :

- **Parseurs DAX et Power Query** (temps 2).
- **Normalisation plus poussée** (à l'usage).
- **Mode hébergé ou partagé** pour les résultats de comparaison (à l'usage).
- **Mode planifié et comptes de service** (beaucoup plus tard).
- **Tables de référence de paramètres** (selon la maturité des clients).